JOHANNES KEPLER
UNIVERSITY LINZ

Functional Verification with cocotb



Simon Dorrer

KV VLSI Design, 336.051, WiSe 26/27

INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at

Today's Goals

After this lecture, you can

- write a **verification plan** with checks and coverage goals
- build a testbench from **driver, monitor, model, scoreboard**
- choose **directed, exhaustive, or random** stimuli
- measure **code and functional coverage**
- write **assertions** and know where tools support them
- check your checks with **mutation testing**
- run the same testbench on the **netlist**, also with **SDF delays**



Chapter 4

Lecture notes: Chapter 4 of iic-jku.github.io/vlsi-design

Why Verification Matters



A Chip Cannot Be Patched

- A bug in the masks is in **every die** of the wafer
- The fix is a **new tapeout**: masks, fab run, packaging, test
  - months, and millions of euros in advanced technologies
- The cost of a bug grows with every step of the flow
  - RTL: minutes and a new simulation
  - netlist or layout: another run of the flow
  - silicon: a new tapeout
- Verification often takes **more effort than the RTL**

Verification, Test, Validation

	Question	When
verification	Does the design meet its specification?	before tapeout, on models
test	Is this die free of manufacturing defects?	every die, after fabrication
validation	Does the chip do what the application needs?	first silicon, on a board

- Today: **functional verification**
- Test and design for test: Chapter 9

Methods and What They Answer

Method	Answers	Limit	Course
simulation	correct for the applied stimuli?	only those stimuli	today
equivalence checking	netlist = RTL?	no timing	Chapter 5
static timing analysis	every path in time?	constraints only, no function	Chapter 6
formal property checking	property for all input sequences?	small blocks	outlook
FPGA prototyping	runs the real software?	needs hardware	outlook

- Simulation shows that a bug **exists**, never that **none is left**

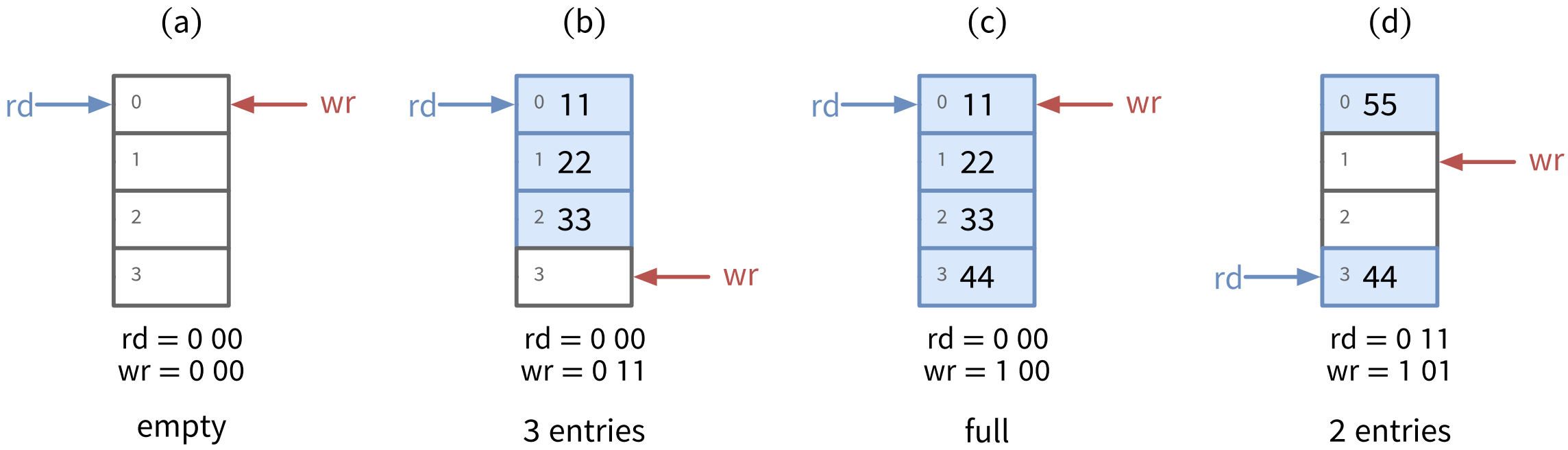
A FIFO and Its Verification Plan



Specification of the FIFO

- **4 entries** of 8 bits, first-word fall-through: `data_o` shows the **oldest** entry
- `push_i` stores `data_i`, but only if **not full**
- `pop_i` removes the oldest entry, but only if **not empty**
- Push and pop in one cycle: **each follows its own rule**
  - full FIFO: the pop removes, the **push is refused**
  - empty FIFO: the push stores, the **pop is refused**
- `data_o` is **undefined** while the FIFO is empty

Pointers with a Wrap Bit



- Pointers have **one bit more** than the address
- Equal pointers: **empty**. Same address, different wrap bit: **full**

Verification Plan

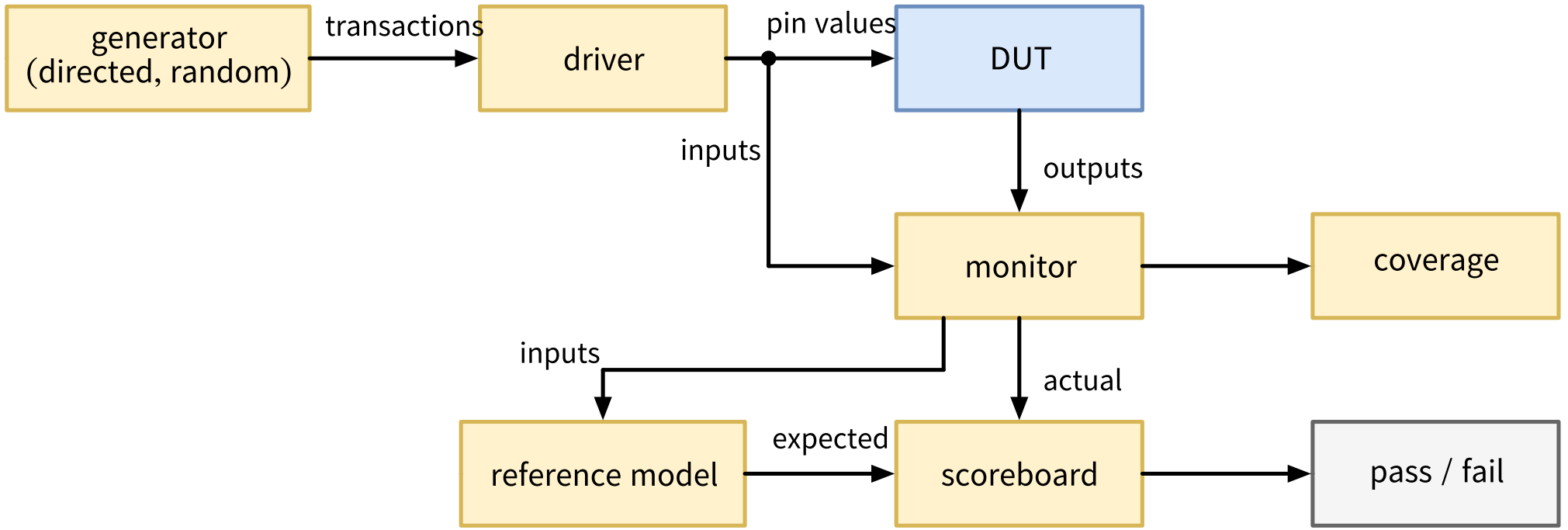
Feature	Check	Coverage goal
order of the entries	scoreboard, every cycle	push and pop in every state
flags after reset, full, empty	scoreboard, every cycle	empty, partial, full reached
full FIFO refuses a push	scoreboard, a_room, a_level	push (and pop) when full
empty FIFO refuses a pop	scoreboard, a_level	pop (and push) when empty
oldest entry stays until popped	scoreboard, a_head	no pop, partial and full
netlist = RTL	same testbench at gate level	all of the above

- Done when **all checks pass** and **all goals are reached**

Anatomy of a Testbench



Testbench Architecture



- **Transactions** instead of pin values, a **reference model** from the specification
- Industry: **UVM** (IEEE 1800.2), in Python: cocotb, pyuvm

Reference Model and Scoreboard

```
class Scoreboard:
    def __init__(self):
        self.model = deque()  # oldest entry first
        self.coverage = Counter()

    def check(self, full, empty, data):
        n = len(self.model)
        assert full == (n == DEPTH), f"full_o = {full}, {n} entries"
        assert empty == (n == 0), f"empty_o = {empty}, {n} entries"
        if n > 0:
            assert data == self.model[0], f"data_o = {data}, ..."

    def update(self, push, data, pop):
        full, empty = len(self.model) == DEPTH, len(self.model) == 0
        ...                     # count the coverage bin
        if pop and not empty:
            self.model.popleft()
        if push and not full:  # a full FIFO refuses a push
            self.model.append(data)
```

- No pointers, no wrap bit: an **independent** model

Driver and Monitor

```
async def driver(dut, transactions):
    for push, data, pop in transactions:
        dut.push_i.value = push
        dut.data_i.value = data
        dut.pop_i.value = pop
        await FallingEdge(dut.clk_i)

async def monitor(dut, sb):          # started with cocotb.start_soon()
    while True:
        await ReadOnly()              # all signals settled, after the driver
        sb.check(dut.full_o.value, dut.empty_o.value, dut.data_o.value)
        sb.update(int(dut.push_i.value), int(dut.data_i.value),
                  int(dut.pop_i.value))
        await FallingEdge(dut.clk_i)
```

- Outputs: the state **after** the last rising edge
- Inputs: the values **for** the next rising edge

Live Demo: The FIFO Testbench

```
cd content/verification/code/fifo/testbenches/cocotb
python3 fifo_tb.py                                  # Icarus Verilog
COCOTB_RANDOM_SEED=1 python3 fifo_tb.py             # reproducible
SIM=verilator python3 fifo_tb.py                    # with code coverage
```

- Two tests: **directed** fill and drain, **constrained random**
- Each prints its **coverage bins**

Stimulus



Directed, Exhaustive, Random

Stimulus	Strength	Weakness
directed	readable, hits known corners	only what the author thought of
exhaustive	a proof (in simulation)	2^n vectors: small blocks only
random	finds the unexpected, cheap	misses rare corners without help

- Best: **directed corner values**, then **constrained-random** values

How Many Vectors?

Design	Input vectors	At 100,000 vectors/s
16-bit multiplier	$2^{32} \approx 4.3 \cdot 10^{9}$	about half a day
32-bit adder	$2^{64} \approx 1.8 \cdot 10^{19}$	almost six million years
FIFO, 10 cycles	$1024^{10} \approx 1.3 \cdot 10^{30}$	out of reach

- Sequential designs: the inputs **and their history**
- Exhaustive only for small blocks and **small parameters**

Seeds and Constraints

```
for _ in range(20):                          # 20 phases ...
    p_push = random.choice([0.2, 0.5, 0.8])
    for _ in range(50):                      # ... of 50 cycles
        push = int(random.random() < p_push)
        pop = int(random.random() < 1 - p_push)
```

- cocotb prints the **seed**: rerun with `COCOTB_RANDOM_SEED=<seed>`
- **Constraints** steer toward the corners: phases that fill or drain

Exercise (Section 4.6.3): verify a barrel shifter, exhaustive and random.

Coverage



Code Coverage with Verilator

```
SIM=verilator COCOTB_TEST_FILTER=test_fill_and_drain python3 fifo_tb.py
verilator_coverage --annotate-min 1 --annotate-points --annotate annotated \
  sim_build/coverage.dat
```

Directed test	Coverage
line	100 % (3/3)
branch	100 % (8/8)
expression	100 % (14/14)
toggle	50.7 % (69/136)

- Only **toggle** shows a gap
- Wrap bits **never fall**
- Both tests: toggle 100 %
- Expression point `(full_o==1) => 0`: hit when full, **with or without** a push

Functional Coverage

Directed test	empty	partial	full
idle	2	0	0
push	1	3	0
pop	0	3	1
push+pop	0	0	0

Random, seed 1	empty	partial	full
idle	51	85	54
push	16	142	169
pop	126	123	26
push+pop	47	109	54

- Directed test: **5 of 12** bins, despite 100 % line coverage
- Random test: **all 12**, and it **checks this itself**

What Coverage Does Not Tell

- Coverage measures the **stimuli**, not the **checks**
- A testbench that checks **nothing** reaches the **same coverage**
- Code coverage cannot see a **missing feature**
- Full coverage is **necessary**, not **sufficient**

Assertions



Immediate and Concurrent Assertions

```
assert (!(full_o && empty_o)) else $error("full and empty"); // immediate
a_room: assert property (@(posedge clk_i) disable iff (!rst_ni)
  full_o && pop_i |=> !full_o);                              // concurrent
```

- Concurrent: sampled at the clock edge, values **just before** the edge
- `a |-> b`: b in the **same** cycle, `a |=> b`: in the **next** cycle
- `$stable(x)`, `$past(x)`, `$rose(x)`, `$fell(x)`: sampled values only
- `cover property`: functional coverage in SVA

A Checker for the FIFO

```
assign level = wr_ptr_q - rd_ptr_q;              // white box: pointers
a_flags: assert property (@(posedge clk_i) disable iff (!rst_ni)
  !(full_o && empty_o));
a_level: assert property (@(posedge clk_i) disable iff (!rst_ni)
  32'(level) <= DEPTH);
a_head: assert property (@(posedge clk_i) disable iff (!rst_ni)
  !empty_o && !pop_i |=> $stable(data_o));
a_room: assert property (@(posedge clk_i) disable iff (!rst_ni)
  full_o && pop_i |=> !full_o);
c_full_push_pop: cover property (@(posedge clk_i)
  disable iff (!rst_ni) full_o && push_i && pop_i);
bind fifo fifo_sva #(.WIDTH(WIDTH), .DEPTH(DEPTH)) u_sva (.*);
```

- `bind` adds the checker to **every** `fifo`, the **RTL stays untouched**
- `SIM=verilator SVA=1`: the cover property counts **54** (seed 1), the bin push+pop when full

Tool Support

Tool	Immediate	Concurrent
Icarus Verilog 14.0	yes, but $error exits with 0	error, or dropped silently
Verilator 5.052	yes	yes, on by default
Yosys 0.70	syntax error at else	syntax error at @(posedge
Yosys with slang	read	unsupported for |=>

- Concurrent assertions in a **checker file**, run with **Verilator**
- Checks at the **ports** in the **cocotb testbench**: every simulator, also the netlist

Assertions or Scoreboard?

- Our scoreboard checks every output every cycle: it fires **half a cycle earlier**
- Assertions pay off where a scoreboard is **blind or absent**
  - they check **internal state**, e.g., the pointers
  - they **name the broken rule**
  - they stay active **deep inside a larger design**
- Synthesis drops assertions: only **port checks** survive at gate level
- Outlook: **formal** tools (SymbiYosys) prove assertions for every input

Mutation Testing



Who Checks the Checker?

- Insert a small **deliberate bug**, a **mutant**, and rerun the tests
- A test fails: **killed**. All pass: **survived**
  - a hole in the testbench, or an **equivalent** mutant
- Mutation score: killed / non-equivalent mutants
- Example: `if (push)` → `if (push_i)`
  - a push into a **full** FIFO overwrites the oldest entry

Exercise (Section 4.9): kill six mutants with tests and assertions.

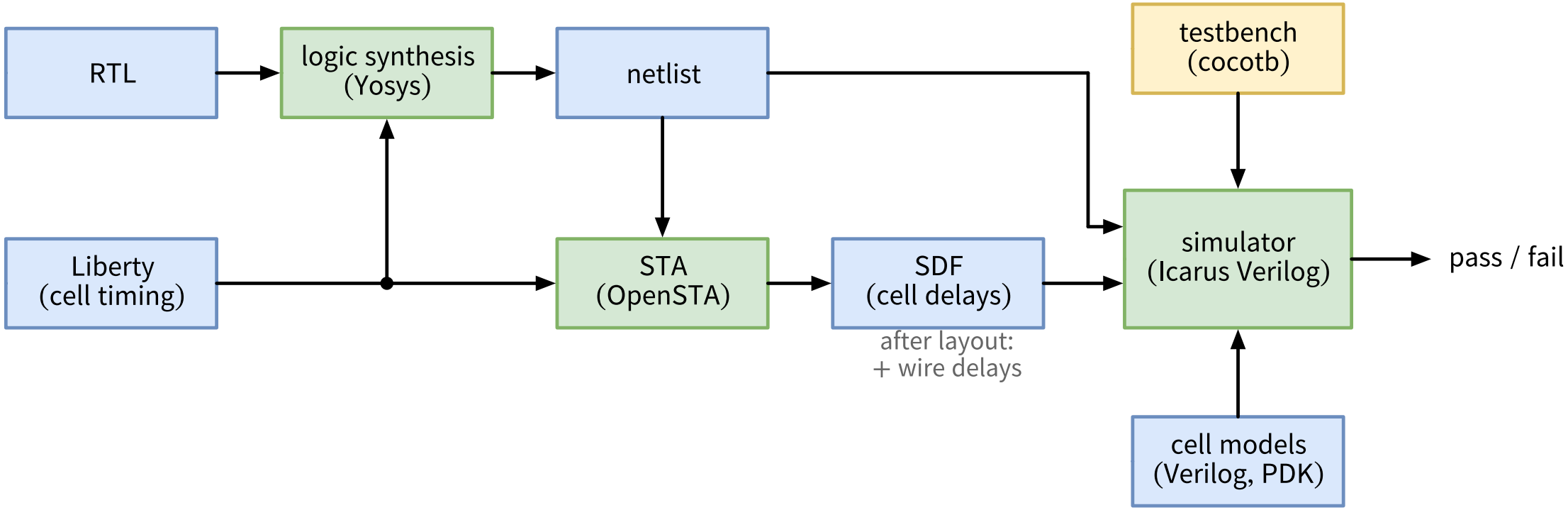
Gate-Level Simulation



Why Simulate the Netlist?

- The chip implements the **netlist** (Chapter 3)
- Equivalence checking proves the functions, but cannot see
  - unknown **start values**, a missing **reset**, propagation of **x**
- Only in the netlist: e.g., **scan chains** (Chapter 9)
- With cell delays: a second, independent look at the **timing**

Ingredients



The Same Testbench on the Netlist

```
cd content/verification/code/fifo
yosys -p "read_verilog -sv rtl/fifo.sv; synth -top fifo; ...; \
  write_verilog -noattr fifo.nl.v"
cd testbenches/cocotb
GL=1 python3 fifo_tb.py                    # cell models, no delays
```

- **103 cells**, 38 flip-flops, 3008 µm²: both tests **pass**
- `data_o` is xxxxxxxx after reset until the first push
  - the storage has **no reset**
  - no failure: the specification leaves `data_o` **undefined** when empty

SDF: Standard Delay Format

```
(CELL
 (CELLTYPE "sg13cmos5l_dfrbpq_1")
 (INSTANCE _120_)
 (DELAY (ABSOLUTE
  (IOPATH CLK Q (0.159:0.161:0.163) (0.155:0.156:0.158))))
 (TIMINGCHECK
  (SETUP (posedge D) (posedge CLK) (0.117:0.119:0.121))
  (HOLD (posedge D) (posedge CLK) (-0.062:-0.064:-0.065))))
```

- Triples **min:typ:max**, IOPATH for a **rising** and a **falling** output
- TIMINGCHECK: setup, hold, recovery, removal, pulse width
- Trap: OpenSTA writes **(min::max)** without `-include_typ`
  - Icarus uses **typ** by default: nothing annotated, **no message**

Timing Simulation with SDF

```
sta -no_init -exit syn/sta.tcl            # writes fifo.sdf
GL=1 SDF=1 python3 fifo_tb.py             # with the cell delays
```

- Both tests pass, with 38 warnings `TIMINGCHECK not supported`
  - Icarus applies the delays, but checks **no setup or hold** time
- Check that a delay **shows up**: `empty_o` falls 0.44 ns after the edge
- 20,000 cycles: 1.0 s RTL, 1.2 s gate level, 1.5 s with delays

Timing Simulation and STA

- Reduce the clock period: where does **STA** fail, where the **simulation**?
- Constraints describe the testbench: inputs and outputs get **half a period**
- Which one do you trust for the **tapeout**?

Exercise (Section 4.10.2): timing simulation against static timing analysis.

Choosing a Simulator

	Icarus Verilog	Verilator
principle	event-driven, engine vvp	compiles to C++, order fixed at build
values	four, shows x	two, replaces x
build of the FIFO	0.01 s	8 s first, then about 2 s
1,000,000 cycles	6 s	0.4 s
cell delays, SDF	read, but no timing checks	ignored (SPECIFYIGN)

- **Verilator**: lint, long runs, code coverage, concurrent assertions, random start values
- **Icarus Verilog**: short RTL runs, x in the RTL, and **every netlist**
- Verilator pays off once an **Icarus run** takes longer than the **Verilator build**

Regression and Summary



Regression and Continuous Integration

- One command, all tests, one verdict: `make sim-all`, `make regression`
- Template CI: `make regression` every night in the IIC-OSIC-TOOLS container
- A failing test must **fail the command**: `get_results()` (Chapter 2)
- Fast tests on every change, gate level and flow **every night**
- **Fixed seeds** for reproducibility, new seeds nightly, always **log the seed**
- Keep it **green**, and turn every bug into a **test**

Summary

- A **verification plan**: a check and a coverage goal per feature
- Driver, monitor, **independent model**, scoreboard, coverage
- **Directed** corners plus **constrained random**, exhaustive only when small
- **Coverage** measures the stimuli, **mutants** measure the checks
- **Assertions** in a bound checker for Verilator, port checks in cocotb
- Same testbench on the **netlist**: a timing simulation does **not replace STA**
- **Icarus** for x and every netlist, **Verilator** for long runs and coverage
- A **regression** fails when any test fails, every change, every night

Exercises

1. Verify a Barrel Shifter (4.6.3)
2. Kill the Mutants (4.9)
3. Timing Simulation Against Static Timing Analysis (4.10.2)

All code in `content/verification/code` of the course repository, every number reproduced by `scripts/check_verification.sh`



Chapter 4



JOHANNES KEPLER
UNIVERSITY LINZ